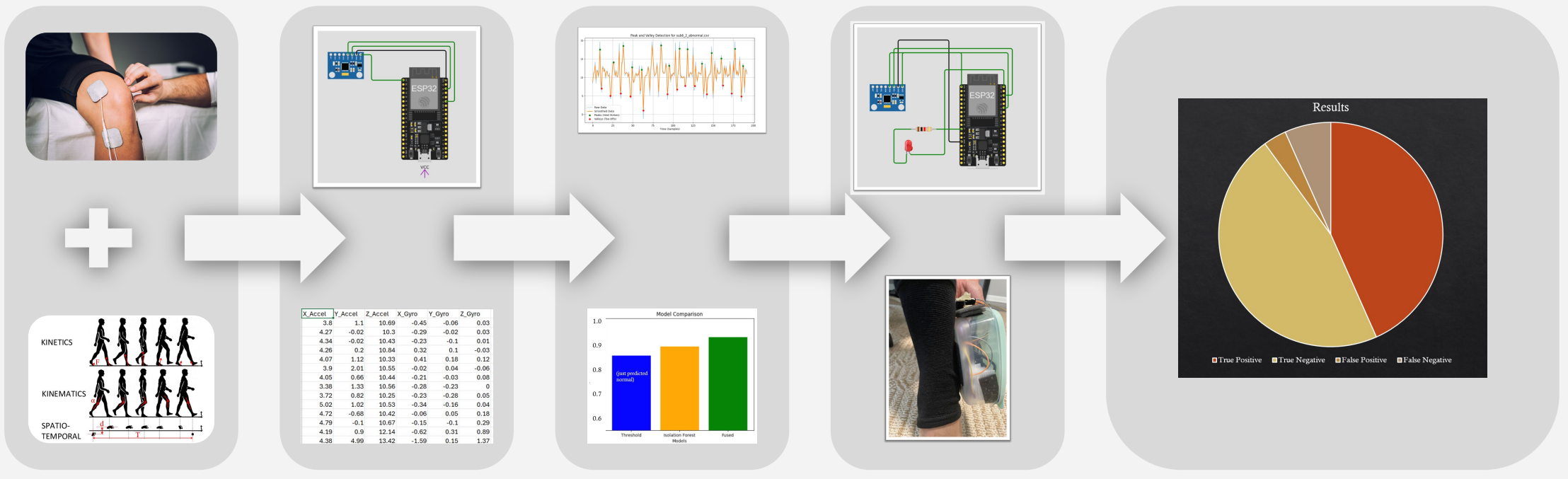
Motion Mend

A Novel Wearable that Utilizes Gait Analysis and Neuromuscular Electrical Stimulation to Optimize Muscle Injury Recovery

Diya Venkataragavan

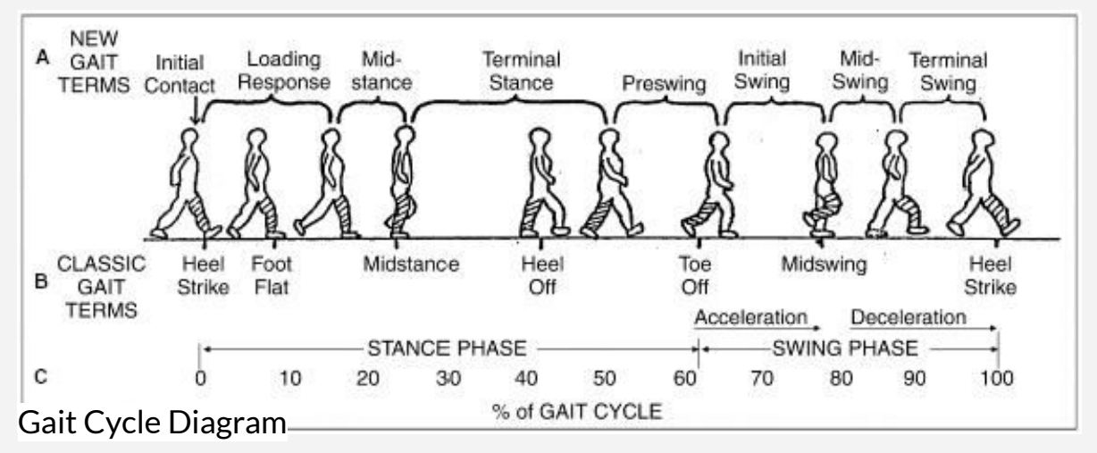
VISUAL OVERVIEW



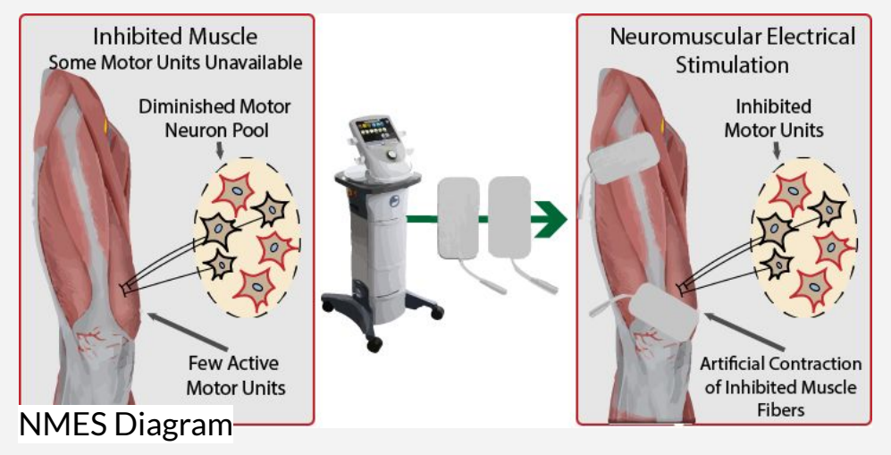
BACKGROUND

GAIT ANALYSIS is the evaluation of human locomotion patterns performed in a research or clinical setting.
It is used to diagnose conditions and further advancements within physical therapy and orthopedics. Although traditionally performed by professionals through manual approaches and visual observation, recent advancements in motion capture and wearable technologies have opened the door for more digitized practices.

The **GAIT CYCLE** refers to the sequence of movements that occur when walking, from one step to the next.
It is divided into two main phases: the stance phase and the swing phase. The stance phase occurs when the foot is in contact with the ground and bears weight, while the swing phase happens when the foot is lifted off the ground and moves forward. These two phases alternate with each step.



Gait Cycle Diagram


Inhibited Muscle
Some Motor Units Unavailable
Diminished Motor Neuron Pool
Few Active Motor Units
Neuromuscular Electrical Stimulation
Inhibited Motor Units
Artificial Contraction of Inhibited Muscle Fibers

NMES Diagram

NEUROMUSCULAR ELECTRICAL STIMULATION (NMES) is a rehabilitation practice. It involves directing electric impulses toward a target area of the body to stimulate weak or paralyzed muscle. NMES-combined training has significant benefits over traditional exercise, but it's untargeted approach causes muscle fatigue and potential damage after long periods of use.

PROBLEM FRAMING

Expensive, inaccessible lower-limb diagnosis and analysis tools are impractical for everyday use.

Lower-limb orthopedic recovery regimes are lengthy, grueling, and difficult for the patient.

Lower-limb recovery devices don't adapt with the user, making adjustment periods inefficient and expensive.

RESEARCH OBJECTIVES

The **SOLUTION** is Motion Mend, a novel wearable that combines gait analysis and NMES to streamline lower-limb injury recovery. Using a microcontroller and IMU, it records gait data, applies mathematical and machine learning models for classification, and triggers electrical stimulation when gait abnormalities are detected. By segmenting the gait cycle into stance and swing phases, Motion Mend precisely targets abnormalities, activating stimulation only when the affected muscle is in use and circumventing fatigue and muscle damage.

Create an inexpensive, accessible diagnosis and analysis tool by adapting an existing, practical invention.

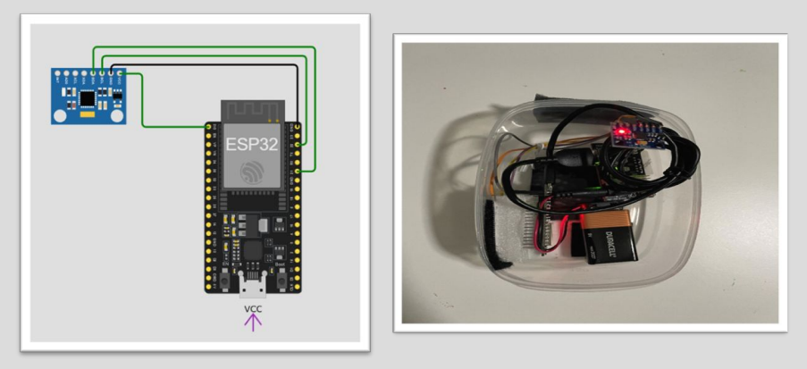
Invent a device that shortens recovery length and makes the process easier.

Construct a program that continuously adapts to the patients improving gait over time.

Motion Mend **SOLVES** by making NMES adaptive to real-time gait patterns, enabling broader, accessible applications in orthopedics and physical therapy while reducing the time, cost, and effort of recovery.

ENGINEERING PROCEDURE

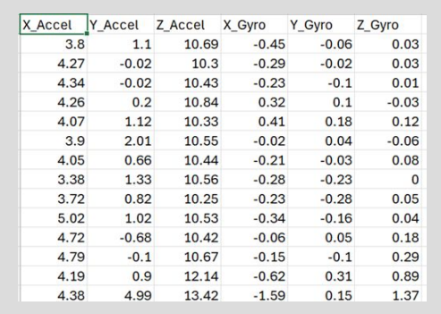
HARDWARE CONFIGURATION AND PROGRAMMING



A hardware device was assembled with an MPU-6050 sensor, ESP32 microcontroller, and power supply. The MPU-6050 measured motion data, which the ESP32 collected and transmitted to a computer. Everything was housed in a plastic container. A C++ program was downloaded to the ESP32, instructing it to connect to Wifi, start a WebSocket server, and transmit motion data for 20 seconds. A Python program was run on a computer instructing it to connect to the ESP32's WebSocket server, initiate data collection, and save the data in CSV files. The WebSocket connection could be gracefully opened and closed, allowing for multiple run-throughs.

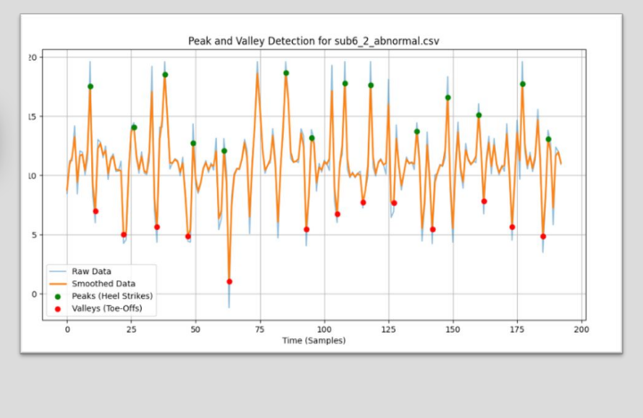
DATASET COLLECTION AND COMPILATION

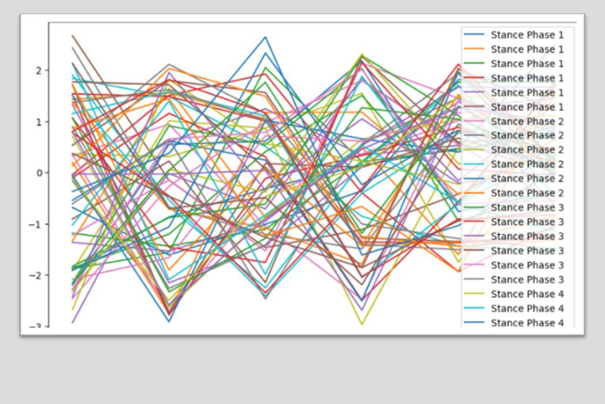
Data was collected on normal and abnormal gait from 10 volunteers using a hardware device: 19 normal, 5 abnormal stance, and 6 abnormal swing run-throughs. Volunteers, 64 inches tall with average inseams of 0.84 meters, wore the device on their left ankles for 20 seconds per run. Normal runs required regular walking, while abnormal runs involved limping: the left leg for stance phase abnormalities, and the right leg for swing phase abnormalities. This setup ensured the NMES device stimulated the correct leg phase without over-straining muscles.



X_Accel	Y_Accel	Z_Accel	X_Gyro	Y_Gyro	Z_Gyro
3.8	1.1	10.69	-0.45	-0.06	0.03
4.27	-0.02	10.3	-0.29	-0.02	0.03
4.34	-0.02	10.43	-0.23	-0.1	0.01
4.26	0.2	10.84	0.32	0.1	-0.03
4.07	1.12	10.33	0.41	0.18	0.12
3.9	2.01	10.55	-0.02	0.04	-0.06
4.05	0.66	10.44	-0.21	-0.03	0.08
3.38	1.33	10.56	-0.28	-0.23	0
3.72	0.82	10.25	-0.23	-0.28	0.05
5.02	1.02	10.53	-0.34	-0.16	0.04
4.72	-0.68	10.42	-0.06	0.05	0.18
4.79	-0.1	10.67	-0.15	-0.1	0.29
4.19	0.9	12.14	-0.62	0.31	0.89
4.38	4.99	13.42	-1.59	0.15	1.37

PEAK DETECTION AND GAIT SEGMENTATION

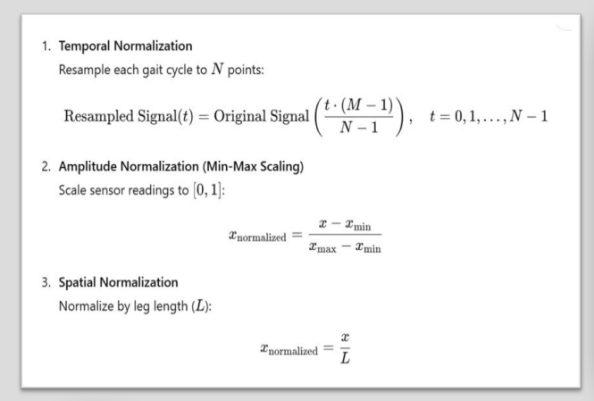




Scikit Learn's peak detection library was used to identify heel strikes and toe-offs by analyzing vertical acceleration (Z_Accel) data. High peaks marked heel strikes, and low points indicated toe-offs. A function was created to segment gait cycles into stance and swing phases by calculating time intervals between peaks. The stance phase was from heel strike to toe-off, and the swing phase was from toe-off to the next heel strike.

DATA NORMALIZATION

After phase segmentation, normalization techniques smoothed noise and increased accuracy, preventing inconsistencies from variations in walking speed, sensor placement, or individual characteristics. The dataset was standardized using temporal resampling, amplitude normalization (e.g., Min-Max scaling), and spatial normalization (e.g., leg length scaling). The normalized stance and swing phase dataset was saved to a new folder. Each phase was saved as a separate CSV file, and files were grouped into separate folders for each run-through.



1. Temporal Normalization
Resample each gait cycle to N points:

$$\text{Resampled Signal}(t) = \text{Original Signal}\left(\frac{t\cdot(M-1)}{N-1}\right),\quad t=0,1,\ldots,N-1$$

2. Amplitude Normalization (Min-Max Scaling)
Scale sensor readings to $[0,1]$:

$$x_{\text{normalized}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

3. Spatial Normalization
Normalize by leg length (L):

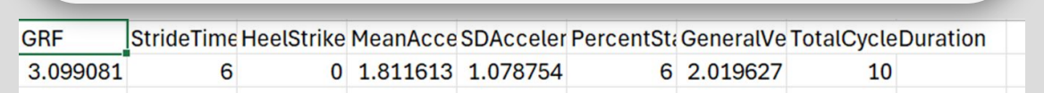
$$x_{\text{normalized}} = \frac{x}{L}$$

```
Dataset/
|---- Runthrough_1/
|         | ---- Phase_1.csv
|         | ---- Phase_2.csv
|         |___ ...
|---- Runthrough_2/
|         | ---- Phase_1.csv
|         | ---- Phase_2.csv
|         |___ ...
|___
```

FEATURE EXTRACTION

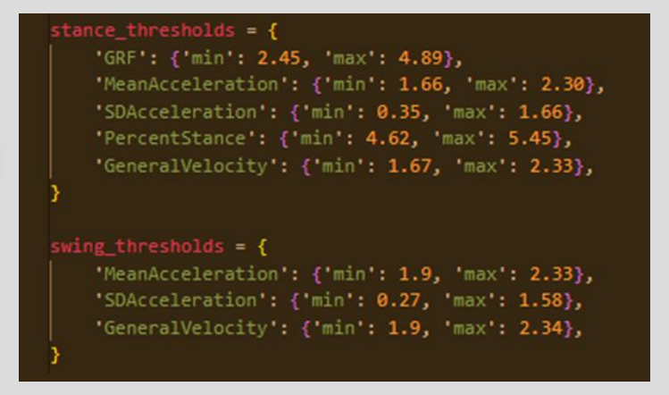
Gait features, derived from accelerometer and gyroscope data, quantify locomotion characteristics like timing, velocity, and stability. These metrics (e.g., velocity, stride time, angular motion) are crucial for machine learning models, enhancing prediction accuracy. Features extracted for stance and swing phases include GRF, stride time, heel strike, mean acceleration, standard deviation, percent stance/swing, general velocity, and total cycle duration. A function iterated over each phase and saved its features to a new file

GRF (Ground Reaction Force): GRF = m × (g + z_accel)
Mean Acceleration: μ_a = (1/n) × Σ a_i
Standard Deviation of Acceleration: σ_a = √((1/n) × Σ (a_i − μ_a)²)
General Velocity: v = ∫ a dt
Angular Motion: θ = ∫ ω dt, where ω is gyroscope data.



GRF	StrideTime	HeelStrike	MeanAcce	SDAcceler	PercentSt	GeneralVe	TotalCycleDuration
3.099081	6	0	1.811613	1.078754	6	2.019627	10

THRESHOLD MODEL



```
stance_thresholds = {
    'GRF': {'min': 2.45, 'max': 4.89},
    'MeanAcceleration': {'min': 1.66, 'max': 2.30},
    'SDAcceleration': {'min': 0.35, 'max': 1.66},
    'PercentStance': {'min': 4.62, 'max': 5.45},
    'GeneralVelocity': {'min': 1.67, 'max': 2.33},
}

swing_thresholds = {
    'MeanAcceleration': {'min': 1.9, 'max': 2.33},
    'SDAcceleration': {'min': 0.27, 'max': 1.58},
    'GeneralVelocity': {'min': 1.9, 'max': 2.34},
}
```

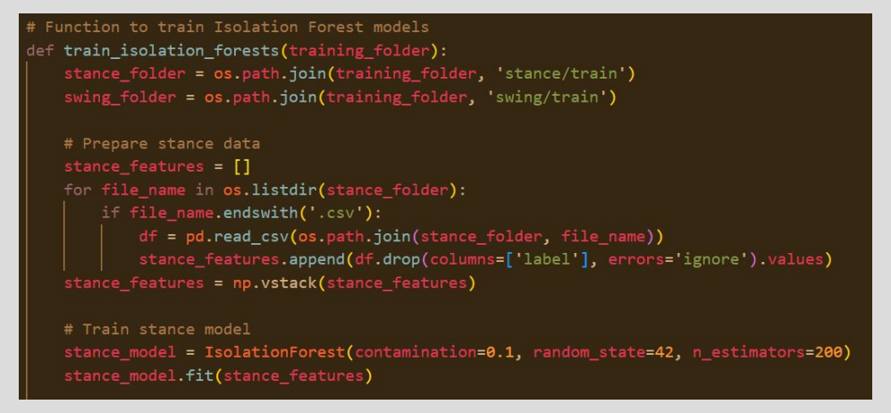
A threshold-based classification system was created to identify normal and abnormal gait phases using domain-specific insights.

Threshold dictionaries for stance and swing phases stored min and max values for abnormalities. A function compared each phase feature value to these thresholds, labeling phases as normal if any feature exceeded abnormal boundaries.

Predictions were saved with 1 for abnormal and 0 for normal. Overall accuracy was 62%, with 88% accuracy for normal phases.

ISOLATION FOREST MODEL

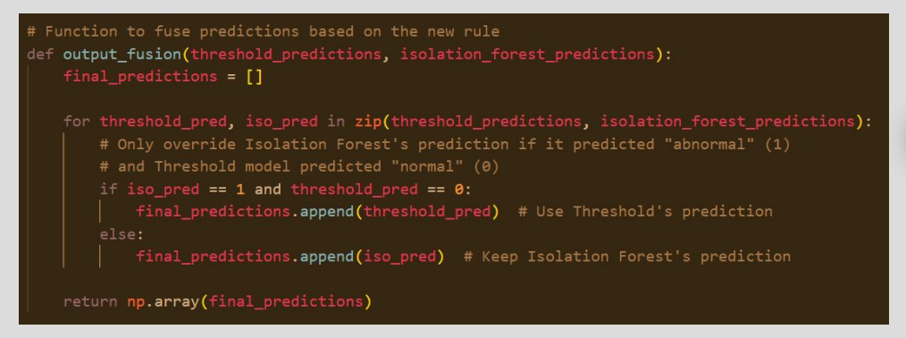
To train the Isolation Forest, data was separated into cumulative datasets for stance and swing phases, then split 85% for training and 15% for testing. Two models were trained—one for stance, one for swing—using ground truth labels in each CSV phase file. The models' results were combined for the final classification report. They accurately identified abnormal gait patterns as outliers in both datasets. Final predictions were saved in each CSV file under a separate column, using the same "0,1" format as the ground truth and threshold labels.



```
# Function to train Isolation Forest models
def train_isolation_forests(training_folder):
    stance_folder = os.path.join(training_folder, 'stance/train')
    swing_folder = os.path.join(training_folder, 'swing/train')

    # Prepare stance data
    stance_features = []
    for file_name in os.listdir(stance_folder):
        if file_name.endswith('.csv'):
            df = pd.read_csv(os.path.join(stance_folder, file_name))
            stance_features.append(df.drop(columns=['label'], errors='ignore').values)
    stance_features = np.vstack(stance_features)

    # Train stance model
    stance_model = IsolationForest(contamination=0.1, random_state=42, n_estimators=200)
    stance_model.fit(stance_features)
```

MODEL OUTPUT FUSION

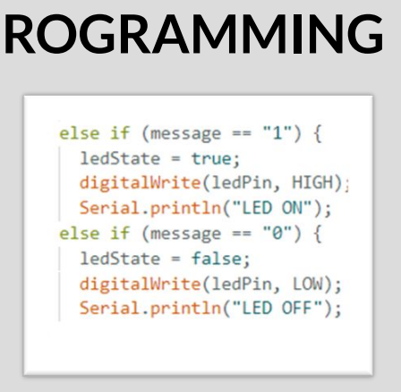
A hybrid prediction function was developed by integrating the threshold-based classification model with the Isolation Forest outputs to enhance system reliability. The fusion process involved overriding Isolation Forest predictions of "abnormal" with "normal" when the threshold model confidently identified a phase as normal. This conservative decision rule was implemented to reduce false negatives, prioritizing scenarios where normal phases were incorrectly flagged as abnormal. Each individual phase of each gait run-through was ultimately assigned a binary label: 1 for abnormal or 0 for normal.



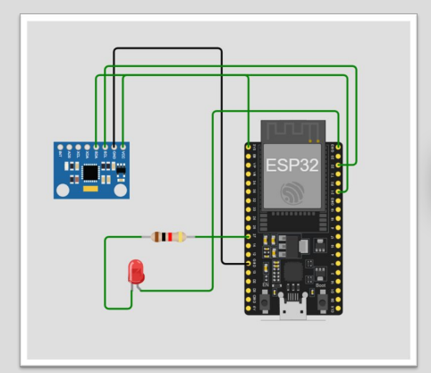
```
# Function to fuse predictions based on the new rule
def output_fusion(threshold_predictions, isolation_forest_predictions):
    final_predictions = []

    for threshold_pred, iso_pred in zip(threshold_predictions, isolation_forest_predictions):
        # Only override Isolation Forest's prediction if it predicted "abnormal" (1)
        # and Threshold model predicted "normal" (0)
        if iso_pred == 1 and threshold_pred == 0:
            final_predictions.append(threshold_pred)  # Use Threshold's prediction
        else:
            final_predictions.append(iso_pred)  # Keep Isolation Forest's prediction

    return np.array(final_predictions)
```

HARDWARE RECONFIGURATION AND REPROGRAMMING

I updated the ESP32 and MPU6050 setup to include an LED simulating an NMES device. The LED lit up during abnormal gait phases, providing a cost-effective alternative to NMES devices. Arduino code was updated for bidirectional communication. The ESP32 transmitted gait data to the computer and received predictions ("1" for abnormal, "0" for normal) via WebSocket. Specific GPIO pins managed LED feedback. The Python pipeline processed incoming data and sent predictions back to the ESP32, enabling real-time feedback and LED activation.



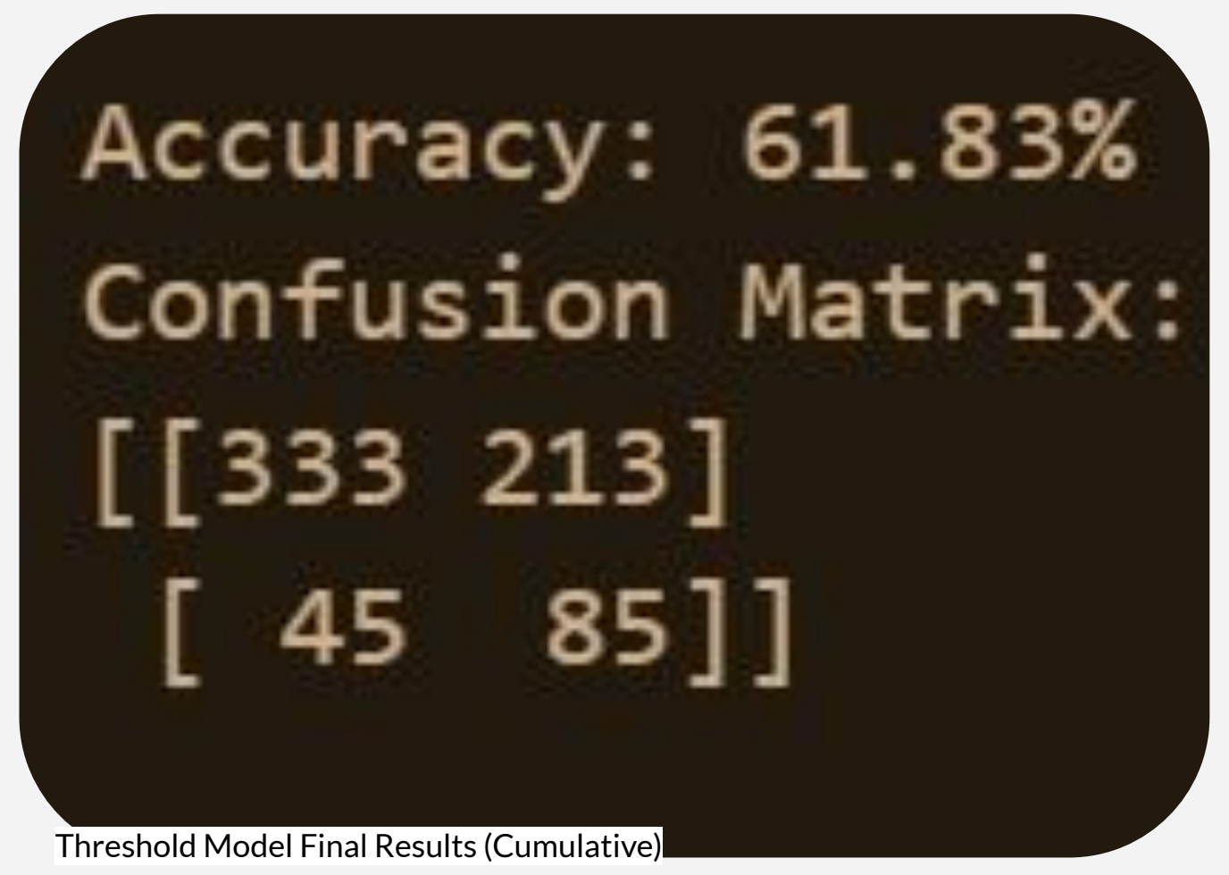
```
else if (message == "1") {
  ledState = true;
  digitalWrite(ledPin, HIGH);
  Serial.println("LED ON");
else if (message == "0") {
  ledState = false;
  digitalWrite(ledPin, LOW);
  Serial.println("LED OFF");
```



FINAL TESTING

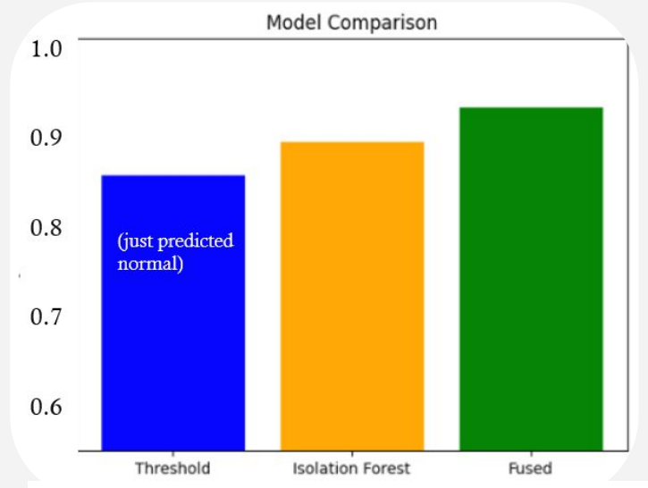
The final pipeline's accuracy was tested over 30 total gait cycles/steps, with one-third of the cycles being normal and two-thirds being abnormal. During half the abnormal cycles the abnormality was faked in the left leg, simulating stance abnormality, and during the other half it was faked in the right leg, simulating swing abnormality. The LED hardware apparatus was monitored during the testing, and its results were recorded simultaneously with my walking. Altogether, the model achieved 90% accuracy, correctly classifying 27 steps and incorrectly classifying 3.

RESULTS AND VALIDATION



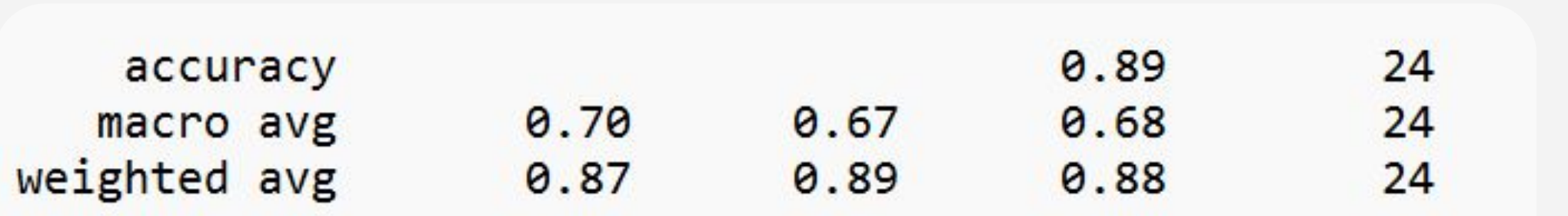
```
Accuracy: 61.83%
Confusion Matrix:
[[333 213]
 [ 45  85]]
```

Threshold Model Final Results (Cumulative)



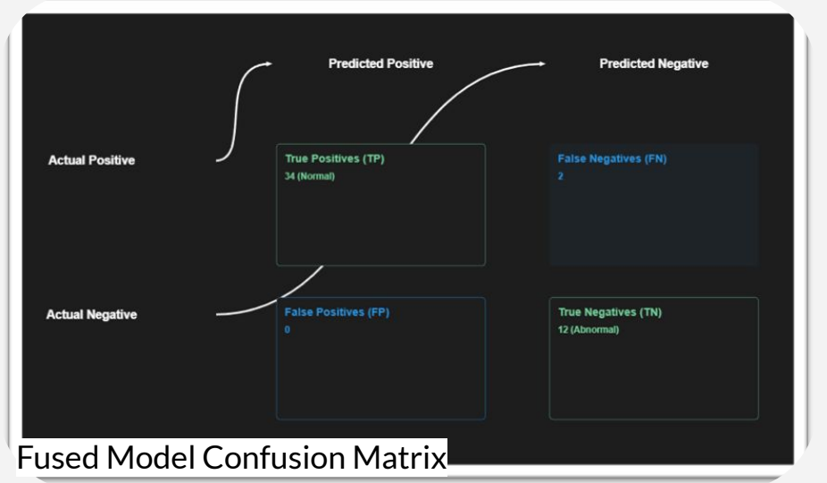
Threshold Model, IsoForest Model, and Fused Results Comparison

NOTE: The threshold model final results were skewed by the predicted abnormal, which were irrelevant in the final fused model.

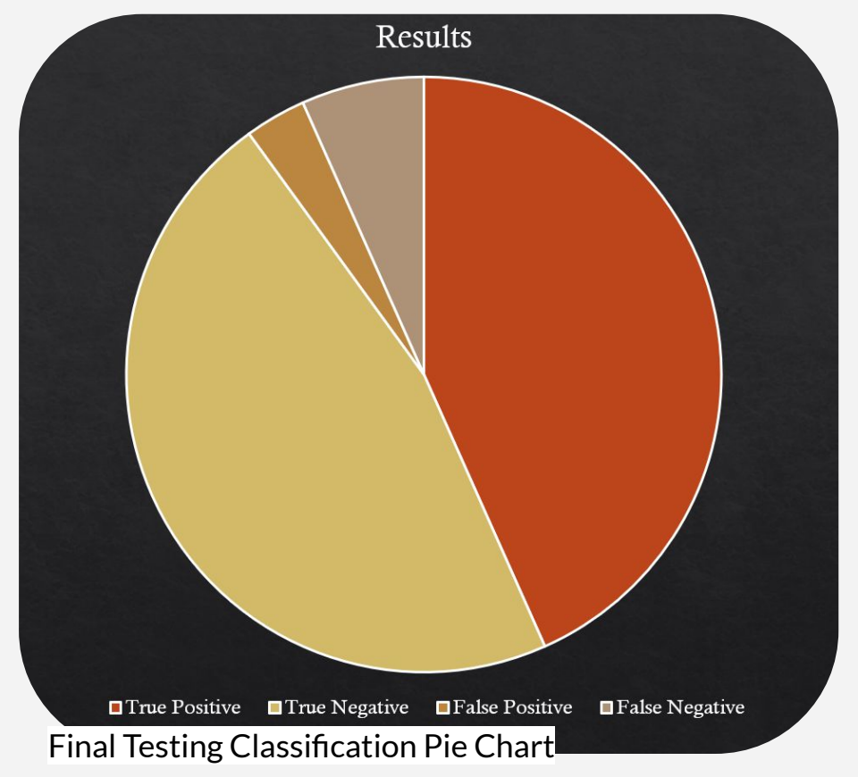


```
    accuracy                         0.89        24
   macro avg       0.70      0.67    0.68        24
weighted avg       0.87      0.89    0.88        24
```

Isolation Forest Model Final Results



Fused Model Confusion Matrix



Final Testing Classification Pie Chart

Classification Report

Total Testing Steps: 30

- **Normal:** 15
- **Abnormal Stance:** 7
- **Abnormal Swing:** 8

Model Performance:

- **Correct Classifications:** 27/30
- **Final Accuracy: 90%**

Misclassifications:

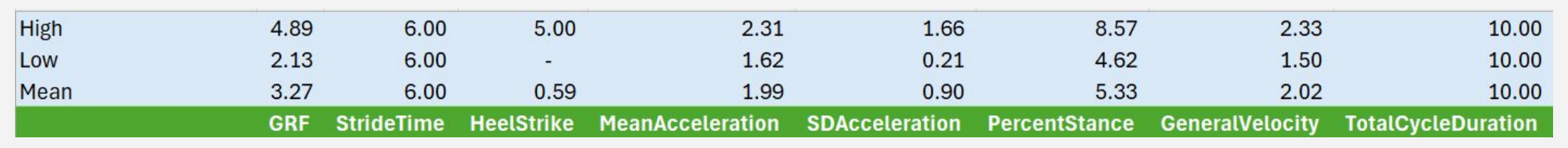
- **2 normal steps** misclassified as abnormal
- **1 abnormal step** misclassified as normal

This indicates a strong overall performance, with a slight tendency toward false positives.

Final Testing Classification Report

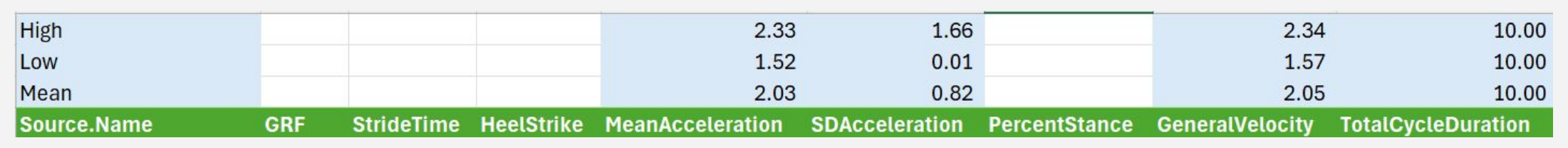
STATISTICAL ANALYSIS

Cumulative Stance Phases (Normal and Abnormal)



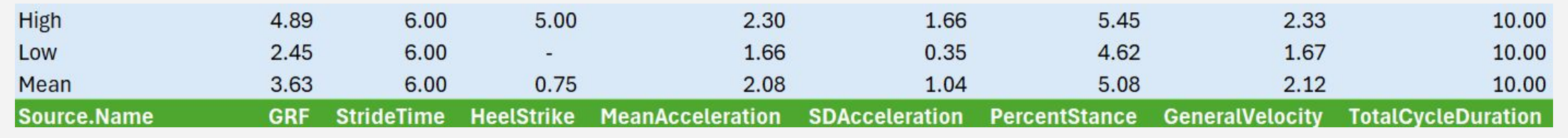
	GRF	StrideTime	HeelStrike	MeanAcceleration	SDAcceleration	PercentStance	GeneralVelocity	TotalCycleDuration
High	4.89	6.00	5.00	2.31	1.66	8.57	2.33	10.00
Low	2.13	6.00	-	1.62	0.21	4.62	1.50	10.00
Mean	3.27	6.00	0.59	1.99	0.90	5.33	2.02	10.00

Cumulative Swing Phases (Normal and Abnormal)



Source.Name	GRF	StrideTime	HeelStrike	MeanAcceleration	SDAcceleration	PercentStance	GeneralVelocity	TotalCycleDuration
High				2.33	1.66		2.34	10.00
Low				1.52	0.01		1.57	10.00
Mean				2.03	0.82		2.05	10.00

Stance Abnormal



Source.Name	GRF	StrideTime	HeelStrike	MeanAcceleration	SDAcceleration	PercentStance	GeneralVelocity	TotalCycleDuration
High	4.89	6.00	5.00	2.30	1.66	5.45	2.33	10.00
Low	2.45	6.00	-	1.66	0.35	4.62	1.67	10.00
Mean	3.63	6.00	0.75	2.08	1.04	5.08	2.12	10.00

CONCLUSIONS

While Neuromuscular Electrical Stimulation (NMES) is highly effective, its untargeted nature introduces flaws that can result in inefficiencies and harm during lower-limb rehabilitation, preventing the device from reaching the main stage. By optimizing Motion Mend, we resolve my research objectives.
Motion Mend addresses this issue by integrating real-time gait analysis with NMES to target stimulation only during abnormal phases of the gait cycle.
The final testing demonstrated high accuracy in detecting abnormal gait phases, validating the device's effectiveness in controlled scenarios.
This innovation has the potential to revolutionize rehabilitation practices by reducing recovery times, lowering costs, and enabling long-term, safe NMES applications in clinical settings.

Create an inexpensive, accessible diagnosis and analysis tool by adapting an existing, practical invention.
|
V
Motion Mend is far cheaper than its alternatives, practical for regular, everyday use by normal people, and adapts NMES to fit a pressing orthopedic need.

Invent a device that shortens recovery length and makes the process easier.
|
V
Motion Mend shortens and streamlines the recovery process by taking a highly effective device without long term applications and adapting it for long-term use.

Construct a program that continuously adapts to the patients improving gait over time.
|
V
Motion Mend's real-time calibration allows it to adapt to any user and their continuously changing recovery process.

FUTURE WORK

1. Data Enrichment - Expanding and improving the representative accuracy of the abnormal gait data pool.
2. Phase Optimization - Increasing the data pipeline's scope of phase analysis. The more phases the pipeline can identify, the more accurate the device.
3. Pipeline Streamlining - Reducing lag and streamlining the analysis process.
4. Real NMES Testing- Wiring the hardware setup through an Op-Amp and an open-circuit NMES device for real testing.
5. Curved Walking and Non-Linear Movement - Non-linear movement involves movements like hopping, cutting and squatting. Adapting this project to classify and stimulate non-linear movement allows Motion Mend to be used 24/7 and during all forms of exercise.

KEY REFERENCES

Hulleck, A. A., Menoth Mohan, D., Abdallah, N., El Rich, M., & Khalaf, K. (2022). Present and future of gait assessment in clinical practice: Towards the application of novel trends and technologies. *Frontiers in medical technology*, *4*, 901331. https://doi.org/10.3389/fmedt.2022.901331

Lake D. A. (1992). Neuromuscular electrical stimulation. An overview and its application in the treatment of sports injuries. *Sports medicine (Auckland, N.Z.)*, *13*(5), 320–336. https://doi.org/10.2165/00007256-199213050-00003

Langley, B., & Greig, M. (2023). The gait abnormality index: A summary metric for three-dimensional gait analysis. *Gait & Posture*, *105*, 87–91. https://doi.org/10.1016/j.gaitpost.2023.07.281

Xu, L., Chen, Y., Tang, L., Chen, J., Tang, J., Yang, X., Su, S., Zhao, C., & Xiao, N. (2024). Accuracy validation of a wearable IMU-based gait analysis in healthy female. *BMC Sports Science, Medicine and Rehabilitation*, *16*(1).

https://doi.org/10.1186/s13102-023-00792-3

Mobbs, R. J., Perring, J., Raj, S. M., Maharaj, M. (2021). Gait metrics analysis utilizing single-point inertial measurement units: a systematic review. *MHealth*, *0*. https://doi.org/10.21037/mhealth-21-17

Santos, G., Wanderley, M., Tavares, T., & Rocha, A. (2022). A multi-sensor human gait dataset captured through an optical system and inertial measurement units. Scientific Data, 9(1), 545. https://doi.org/10.1038/s41597-022-01638-2

More Information

HARDWARE PARTS

ESP-32's are a special kind of microcontroller than has Wifi and Bluetooth capabilities.

MPU-6050 IMU (inertial measurement units) are inertial sensors that contain an accelerometer (sensor that measures speed) and a gyroscope (sensor that measures angular acceleration).

COLLECTION CODE

To utilize the computer as a collector, deliverer, and recipient of information during the data acquisition process, a program was developed and downloaded within Visual Studio Code's Python interface to launch data collection, create and run the WebSocket client, receive and save data into individual CSV files, and handle the graceful opening and closing of the WebSocket connection. The Python program was called within the computer's command prompt, which connected the computer to the ESP32's WebSocket server across their common Wi-Fi network, sent the "START" message to the ESP32, initiated the 20-second collection sequence, and continuously saved the incoming data into an automatically labeled CSV file. Closing the WebSocket connection was done by entering "STOP" or "Ctrl-C" into the command prompt. The program was simply called again to begin a new run-through.

To utilize the ESP32 as a collector, deliverer, and recipient of information during the data acquisition process, a program was developed within the Arduino IDE interface that instructed the ESP32 to connect to the local Wi-Fi, begin a WebSocket server, collect data from the MPU-6050 for 20 seconds, and continuously transmit data through the WebSocket server as it was received. To do this, I first installed the necessary libraries to make the program compatible with the ESP32, MPU-6050, and WebSocket Server. Then, the code was set up to initialize the MPU-6050, connect to Wi-Fi through the pre-specified SSID and password, and open a WebSocket server on the ESP32, which was how the data was transmitted. Subsequently, the data collection process was initiated through the loop function, where the ESP32 was instructed to pull and send the accelerometer and gyroscope data from the MPU-6050 until the collection duration was completed. Along with the two primary functions, the microcontroller was programmed to maintain Wi-Fi connection and handle messages through the WebSocket.

THE DATA

The data itself contained 6 total columns. Each sensor within the IMU has a 3-axis structure, and there are two sensors

1. X Accelerometer - Acceleration on the x-axis
2. Y Accelerometer - Acceleration on the y-axis
3. X Accelerometer - Acceleration on the z-axis
4. X Gyroscope - Angular velocity on the x-axis
5. Y Gyroscope - Angular velocity on the y-axis
6. Z Gyroscope - Angular velocity on the z-axis

Each gait run-through had the same number of datapoints in each column, as each runthrough was collected for the same amount of time.

GAIT FEATURES

·**GRF (Ground Reaction Force):** GRF = m × (g + z_accel)

·**Stride Time:** Calculated as the time difference between consecutive heel strikes.

·**Heel Strike:** Identified using peak detection on vertical acceleration (z_accel).

·**Mean Acceleration:** μ_a = (1/n) × Σ a_i

·**Standard Deviation of Acceleration:** σ_a = √((1/n) × Σ (a_i − μ_a)²)

·**Percent Stance/Swing:** Percent Phase = (Phase Duration / Total Cycle Duration) × 100

·**General Velocity:** v = ∫ a dt

·**Total Cycle Duration:** Calculated as the time difference between the start and end of each gait cycle.

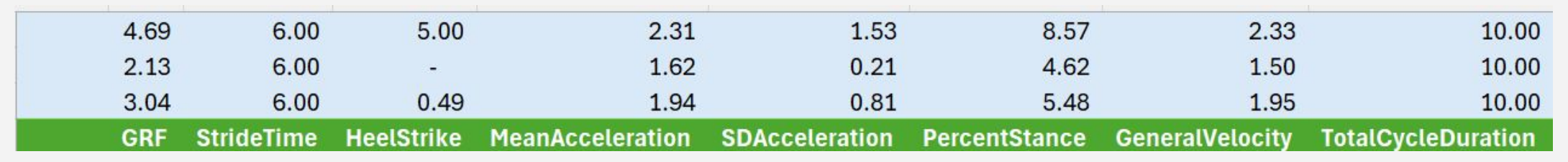
·**Peak Velocity (Swing Phase):** Determined from the maximum velocity within the phase.

·**Angular Motion:** θ = ∫ ω dt, where ω is gyroscope data.

STATISTICAL ANALYSIS

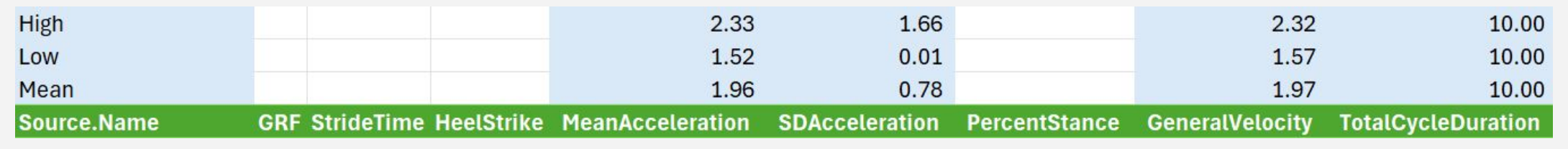
Normal phases showed more feature volatility.

Stance Normal



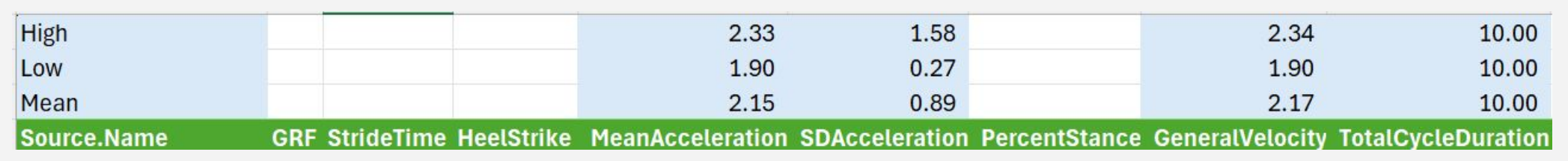
	4.69	6.00	5.00	2.31	1.53	8.57	2.33	10.00
	2.13	6.00	-	1.62	0.21	4.62	1.50	10.00
	3.04	6.00	0.49	1.94	0.81	5.48	1.95	10.00
	GRF	StrideTime	HeelStrike	MeanAcceleration	SDAcceleration	PercentStance	GeneralVelocity	TotalCycleDuration

Swing Normal



High				2.33	1.66		2.32	10.00
Low				1.52	0.01		1.57	10.00
Mean				1.96	0.78		1.97	10.00
Source.Name	GRF	StrideTime	HeelStrike	MeanAcceleration	SDAcceleration	PercentStance	GeneralVelocity	TotalCycleDuration

Swing Abnormal

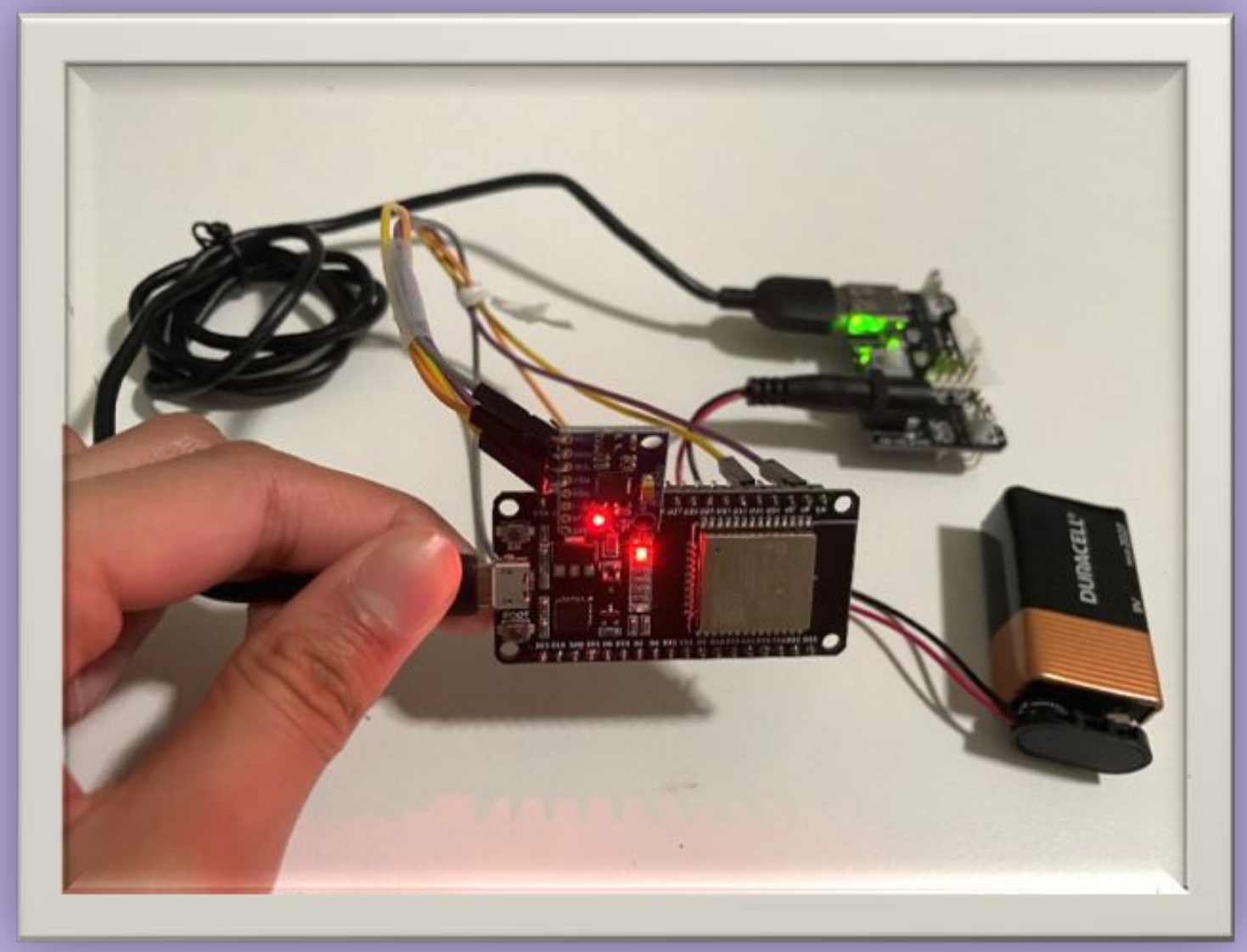


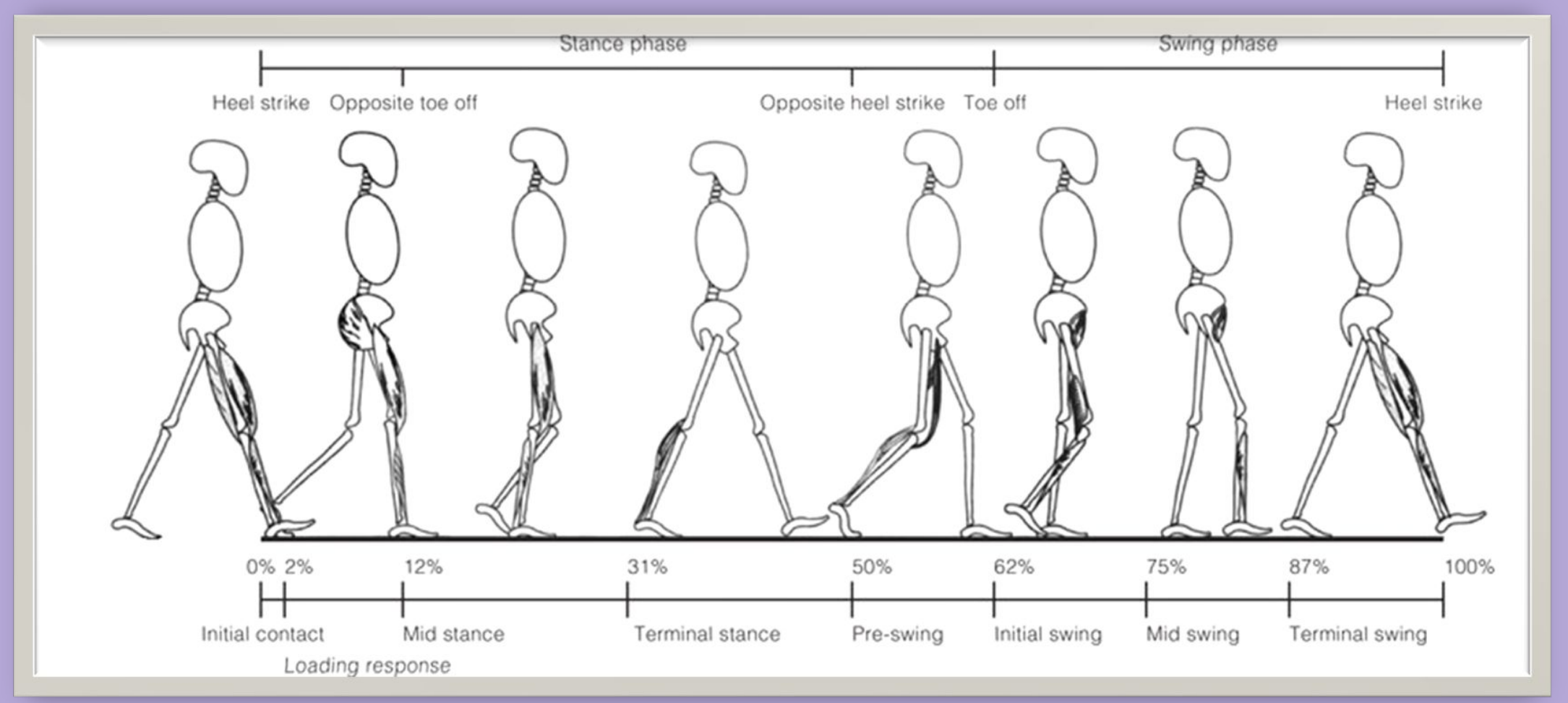
High				2.33	1.58		2.34	10.00
Low				1.90	0.27		1.90	10.00
Mean				2.15	0.89		2.17	10.00
Source.Name	GRF	StrideTime	HeelStrike	MeanAcceleration	SDAcceleration	PercentStance	GeneralVelocity	TotalCycleDuration

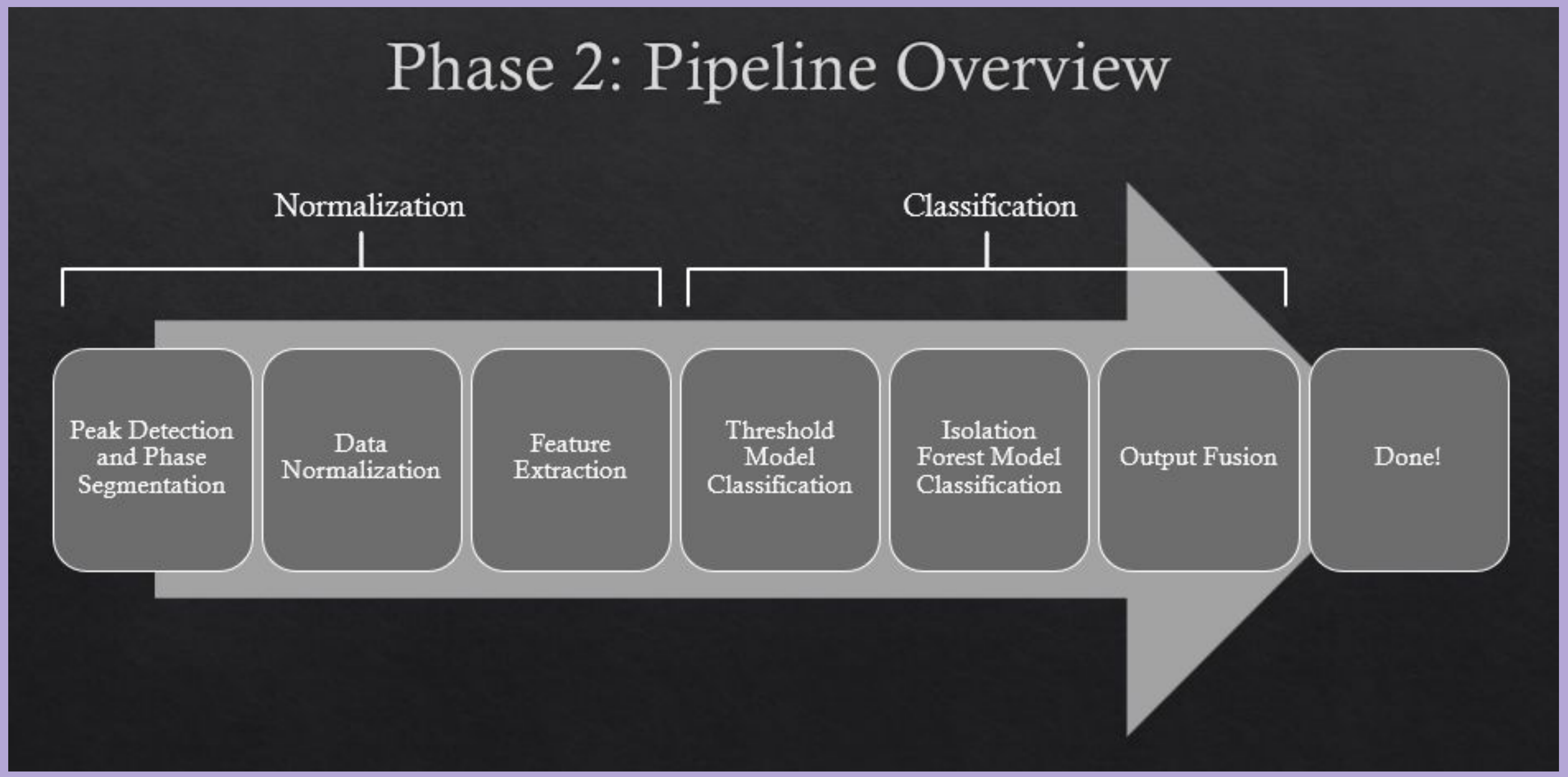
MATERIALS

- ESP-32 Microcontroller
- MPU-6050 Inertial Sensor
- Jumper Wire
- Male-to-Female Dupont Wire
- Breadboard
- Resistor
- Red LED
- Tupperware
- Foam
- Tape
- Ankle Sleeve

PHOTOS, CHARTS, ETC.







Phase 2: Pipeline Overview

Normalization

Classification

Peak Detection and Phase Segmentation

Data Normalization

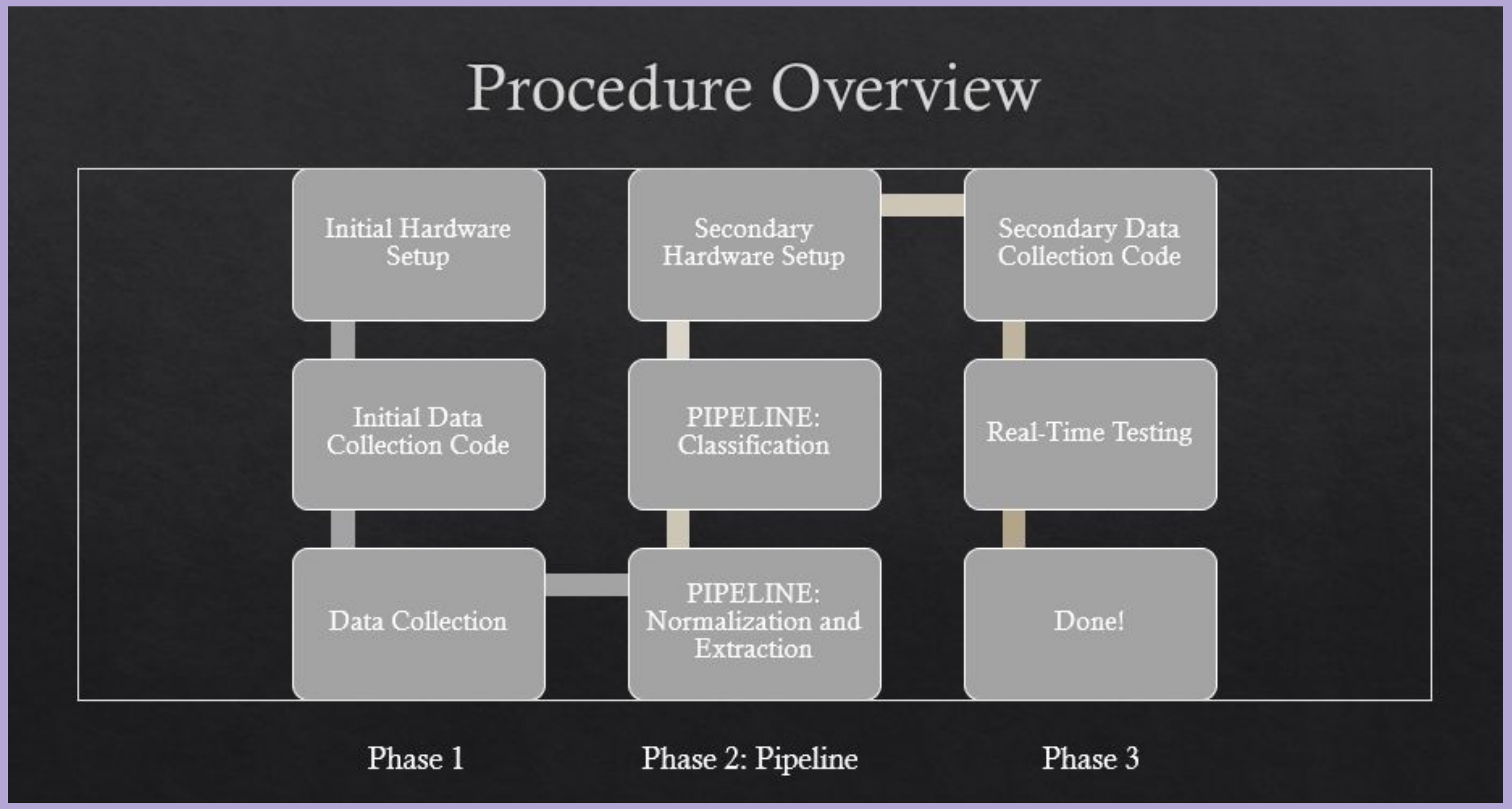
Feature Extraction

Threshold Model Classification

Isolation Forest Model Classification

Output Fusion

Done!



Procedure Overview

Initial Hardware Setup

Initial Data Collection Code

Data Collection

PIPELINE: Normalization and Extraction

PIPELINE: Classification

Secondary Hardware Setup

Secondary Data Collection Code

Real-Time Testing

Done!

Phase 1

Phase 2: Pipeline

Phase 3